PRAKTIKUM 2
Membuat program Aplikasi McCulloch Pitts

NAMA	:	Ihsanta Zaki Sanjaya
KELAS	:	2 D4 Teknik Komputer B
NRP	:	3223600058
DOSEN	:	Riyanto Sigit, ST, M.Kom, Ph.D
MATA KULIAH	:	Praktikum Sistem Cerdas
TANGGAL	:	Rabu, 4 Maret 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 2

Membuat Program Aplikasi McCulloch Pitts

A. Tujuan

1. Mahasiswa dapat memahami dan menjelaskan konsep jaringan syaraf tiruan
2. Mahasiswa dapat menjelaskan model McCulloch Pitts
3. Mahasiswa dapat membuat aplikasi McCulloch

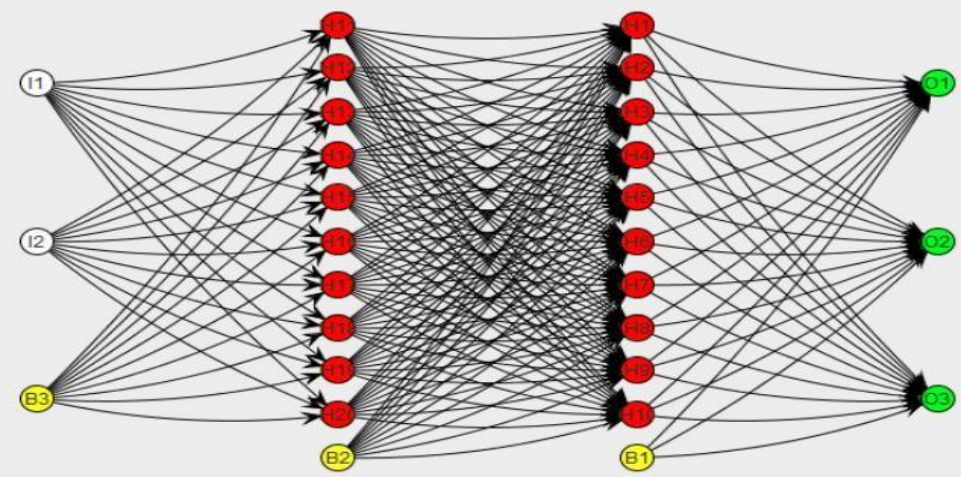
Pitts Software yang diperlukan:

- Microsoft Visual C++
- PyCharm

B. Pendahuluan

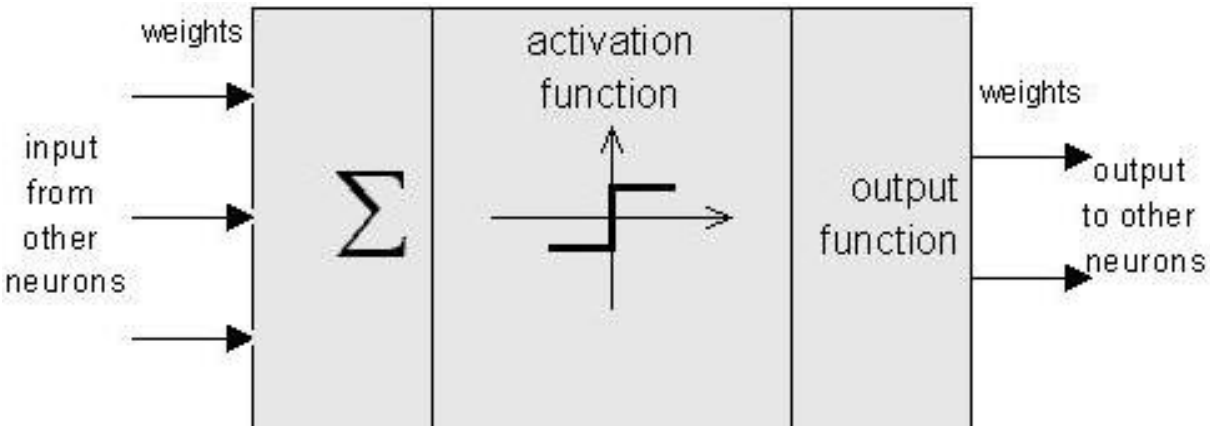
1. Jaringan Syaraf Tiruan

JST adalah sistem pemrosesan informasi yang memiliki karakteristik mirip dengan jaringan syaraf biologi. JST merupakan salah satu upaya manusia untuk memodelkan cara kerja atau fungsi sistem syaraf manusia dalam melaksanakan tugas tertentu. JST mempunyai struktur tersebar paralel yang sangat besar dan mempunyai kemampuan belajar, sehingga bisa melakukan generalisasi, yaitu bisa menghasilkan output yang benar untuk input yang belum pernah dilatihkan.

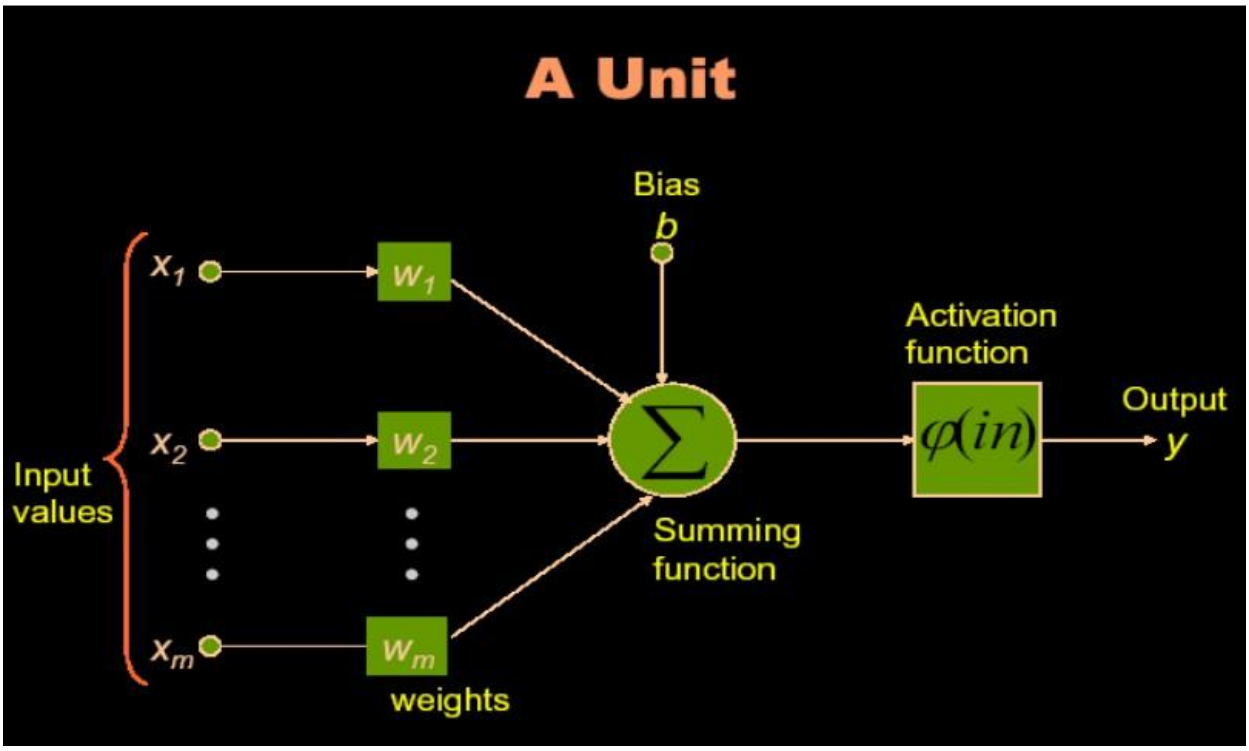


Gambar 1. Pengertian JST

Informasi (sebagai input) dikirim ke neuron melalui suatu pembobotan input. Input diproses oleh fungsi propagation yang menaikan nilai bobot input. Hasilnya dibandingkan dengan threshold oleh activation function. Jika input melampaui threshold, maka neutron akan diaktifkan, jika sebaliknya maka neutron akan inhibit. Jika diaktifkan, neuron akan mengirim output melalui pembobotan output ke neuron lainnya, dan seterusnya.



Gambar 2. Struktur JST



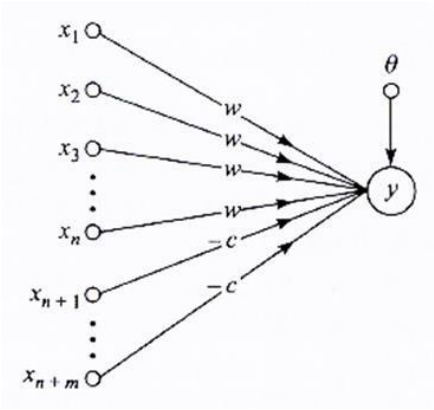
Gambar 3. Pemodelan JST

Sejumlah sinyal masukan x dikalikan dengan masing-masing penimbang atau bobot yang bersesuaian w. Kemudian dilakukan penjumlahan dari seluruh hasil perkalian tersebut dan keluaran yang dihasilkan dilalukan kedalam fungsi pengaktif untuk mendapatkan tingkat derajat sinyal keluarannya F(x,w). Walaupun masih jauh dari sempurna, namun kinerja dari tiruan neuron ini identik dengan kinerja dari sel otak yang kita kenal saat ini. Misalkan ada n buah sinyal masukan dan n buah penimbang, fungsi keluaran dari neuron adalah seperti persamaan berikut:

$$F(x,w) = F(w_1 * x_1 + \ldots + w_n * x_n)$$

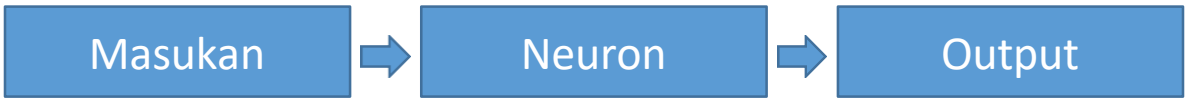
2. McCulloch dan Pitts

McCulloch dan Pitts menyimpulkan bahwa kombinasi beberapa neuron sederhana menjadi sebuah sistem neural akan meningkatkan kemampuan komputasinya. Bobot dalam jaringan yang diusulkan oleh McCulloch dan Pitts diatur untuk melakukan fungsi logika sederhana. Fungsi aktivasi yang dipakai adalah fungsi threshold



Gambar 4. Model Sederhana McCulloch-Pitts

Informasi diproses oleh elemen-elemen sederhana yang disebut neuron. Sinyal-sinyal yang dilewatkan antara neuron saling berhubungan. Setiap sambungan antara dua neuron ada bobotnya masing-masing yang akan mengalikan sinyal yang ditransmisikan. Tiap neuron memiliki fungsi aktivasi yang akan menentukan besaran keluaran.



Gambar 5. Model McCulloch-Pitts

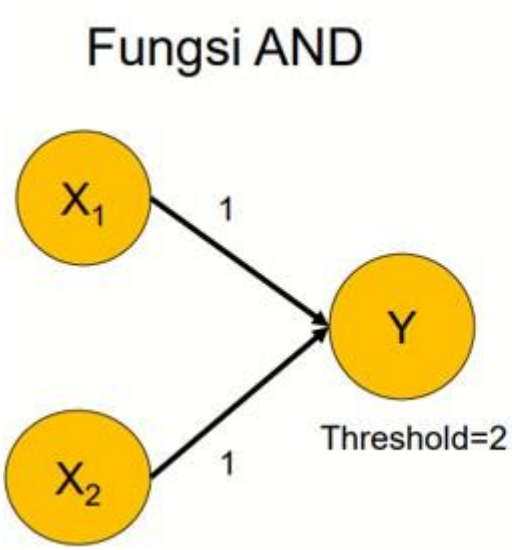
- $u = (w1 * x1) + (w2 * x2)$

$$u \begin{cases} u >= th \Rightarrow y = 1 \\ u < th \Rightarrow y = 0 \end{cases} \text{ if}$$

Gambar 5. Model Matematik

C. **Percobaan**

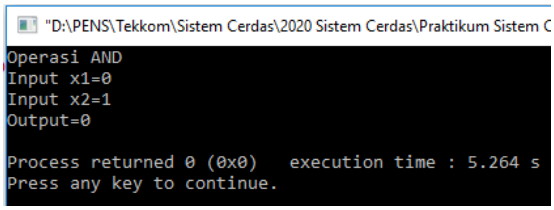
1. Logika AND



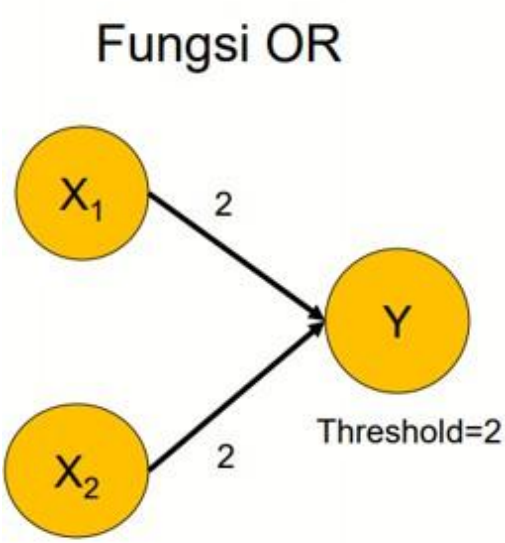
AND

X_1	X_2	Y
0	0	0
0	1	0
1	0	0
1	1	1

```
void main(){
	int w1=1, w2=1, t=2;
	int x1, x2 , y, out;
	printf("Operasi AND\n");
	printf("Input x1=");
	scanf("%d",&x1);
	printf("Input x2=");
	scanf("%d",&x2);
	y=w1*x1+w2*x2;
	if(y>=t)
		out=1;
	else
		out=0;
	printf("Output=%d\n“,out);
}
```



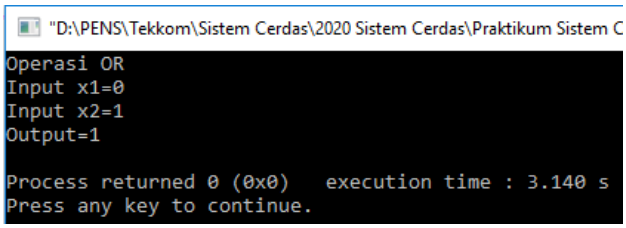
2. Logika OR



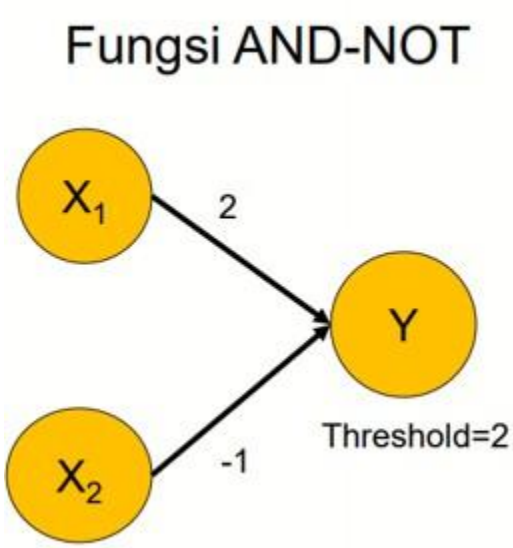
OR

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	1

```
void main(){
    int w1=2, w2=2, t=2;
    int x1, x2, y, out;
    printf("Operasi OR\n");
    printf("Input x1=");
    scanf("%d",&x1);
    printf("Input x2=");
    scanf("%d",&x2);
    y=w1*x1+w2*x2;
    if(y>=t)
        out=1;
    else
        out=0;
    printf("Output=%d\n",out);
}
```



3. Logika AND NOT

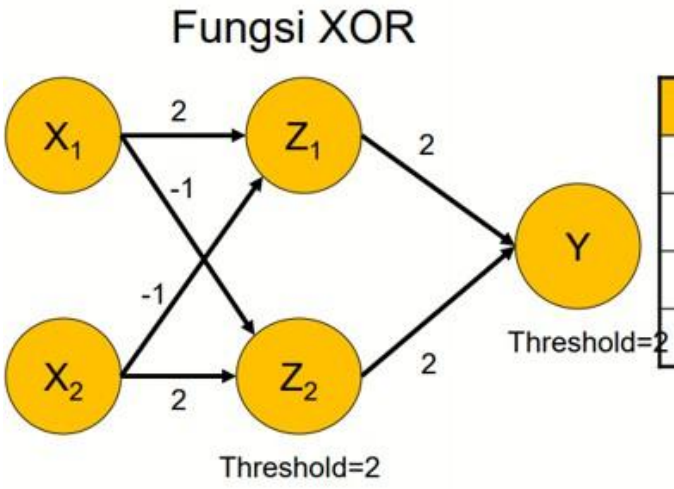


AND-NOT

X_1	X_2	Y
0	0	0
0	1	0
1	0	1
1	1	0

```
void main(){
	int w1=2, w2=-1, t=2;
	int x1, x2, y, out;
	printf("Operasi AND NOT\n");
	printf("Input x1=");
	scanf("%d",&x1);
	printf("Input x2=");
	scanf("%d",&x2);
	y=w1*x1+w2*x2;
	if(y>=t)
		out=1;
	else
		out=0;
	printf("Output=%d\n",out);
}
```

4. Logika XOR

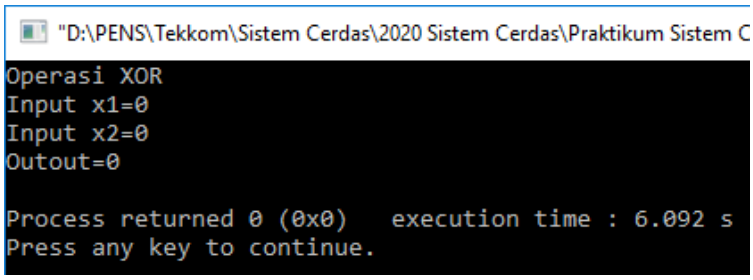


XOR

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	0

X_1 XOR X_2 = (X_1 AND NOT X_2) OR (X_2 AND NOT X_1)

```
void main(){
	int w1=2, w2=-1, w3=2, w4=-1,
	w5=2, w6=2, t=2;
	int x1, x2, y, o1, o2, o3, z1, z2;
	printf("Operasi XOR\n");
	printf("Input x1=");
	scanf("%d",&x1);
	printf("Input x2=");
	scanf("%d",&x2);
	o1=w1*x1+w4*x2;
	if(o1>=t)
		z1=1;
	else
		z1=0;
	o2=w2*x1+w3*x2;
	if(o2>=t)
		z2=1;
	else
		z2=0;
	o3=w5*z1+w6*z2;
	if(o3>=t)
		y=1;
	else
		y=0;
	printf("Outout=%d\n",y);
}
```



D. Hasil Percobaan

1. Logika AND

```c
#include <stdio.h>
void main() {
    int w1=1,w2=1,t=2;
    int x1,x2,y,out;
    printf("Operasi AND\n");
    printf("Input x1=");
    scanf_s("%d", &x1);
    printf("Input x2=");
    scanf_s("%d", &x2);
    y=w1*x1+w2*x2;
    if (y >= t)
        out = 1;
    else
        out = 0;
    printf("Output=%d\n", out);
}
```

Hasil:

a. Input x1 = 0 dan x2 = 0

```
Operasi AND
Input x1=0
Input x2=0
Output=0
```

b. Input x1 = 0 dan x2 = 1

```
Operasi AND
Input x1=0
Input x2=1
Output=0
```

c. Input x1 = 1 dan x2 = 0

```
Operasi AND
Input x1=1
Input x2=0
Output=0
```

d. Input x1 = 1 dan x2 = 1

```
Operasi AND
Input x1=1
Input x2=1
Output=1
```

x1	x2	Output
0	0	0
0	1	0
1	0	0
1	1	1

2. Logika OR

```c
#include <stdio.h>
void main() {
    int w1 = 2, w2 = 2, t = 2;
    int x1, x2, y, out;
    printf("Operasi OR\n");
    printf("Input x1=");
    scanf_s("%d", &x1);
    printf("Input x2=");
    scanf_s("%d", &x2);
    y = w1 * x1 + w2 * x2;
    if (y >= t)
        out = 1;
    else
        out = 0;
    printf("Output=%d\n", out);
}
```

Hasil:

a. Input x1 = 0 dan x2 = 0

```
Operasi OR
Input x1=0
Input x2=0
Hasil=0
```

b. Input x1 = 0 dan x2 = 1

```
Operasi OR
Input x1=0
Input x2=1
Hasil=1
```

c. Input x1 = 1 dan x2 = 0

```
Operasi OR
Input x1=1
Input x2=0
Hasil=1
```

d. Input x1 = 1 dan x2 = 1

```
Operasi OR
Input x1=1
Input x2=1
Hasil=1
```

x1	x2	Output
0	0	0
0	1	1
1	0	1
1	1	1

3. Logika AND NOT

```c
#include <stdio.h>
void main() {
    int w1 = 2, w2 = -1, t = 2;
    int x1, x2, y, out;
    printf("Operasi AND NOT\n");
    printf("Input x1=");
    scanf_s("%d", &x1);
    printf("Input x2=");
    scanf_s("%d", &x2);
    y = w1 * x1 + w2 * x2;
    if (y >= t)
        out = 1;
    else
        out = 0;
    printf("Output=%d\n", out);
}
```

Hasil:

a. Input x1 = 0 dan x2 = 0

```
Operasi AND NOT
Input x1=0
Input x2=0
Output=0
```

b. Input x1 = 0 dan x2 = 1

```
Operasi AND NOT
Input x1=0
Input x2=1
Output=0
```

c. Input x1 = 1 dan x2 = 0

x1	x2	Output
0	0	0
0	1	0
1	0	1
1	1	0

```
Operasi AND NOT
Input x1=1
Input x2=0
Output=1
```

d. Input x1 = 1 dan x2 = 1

```
Operasi AND NOT
Input x1=1
Input x2=1
Output=0
```

4. Logika XOR

```c
#include <stdio.h>
void main() {
    int w1=2,w2=-1,w3=2,w4=-1,w5=2,w6=2,t=2;
    int x1, x2, y, o1,o2,o3,z1,z2;
    printf("Operasi XOR\n");
    printf("Input x1=");
    scanf_s("%d", &x1);
    printf("Input x2=");
    scanf_s("%d", &x2);
    o1 = w1 * x1 + w4 * x2;
    if (o1 >= t)
        z1 = 1;
    else
        z1 = 0;
    o2 = w2 * x1 + w3 * x2;
    if (o2 >= t)
        z2 = 1;
    else
        z2 = 0;
    o3 = w5 * z1 + w6 * z2;
    if (o3 >= t)
        y = 1;
    else
        y = 0;
    printf("Output=%d\n", y);
}
```

Hasil:

a. Input x1 = 0 dan x2 = 0

```
Operasi XOR
Input x1=0
Input x2=0
Output=0
```

b. Input x1 = 0 dan x2 = 1

```
Operasi XOR
Input x1=0
Input x2=1
Output=1
```

c. Input x1 = 1 dan x2 = 0

```
Operasi XOR
Input x1=1
Input x2=0
Output=1
```

d. Input x1 = 1 dan x2 = 1

```
Operasi XOR
Input x1=1
Input x2=1
Output=0
```

x1	x2	Output
0	0	0
0	1	1
1	0	1
1	1	0

E. Tugas

1. Buatlah model neuron McP untuk menyatakan fungsi logika “AND” 3 Masukan
2. Buatlah model neuron McP untuk menyatakan fungsi logika “OR” 3 Masukan
3. Buatlah model neuron McP untuk menyatakan fungsi logika “XOR” 3 Masukan

1. Logika AND 3 Masukan

```c
#include <stdio.h>
int main() {
    int x1, x2, x3;
    int w1 = 1, w2 = 1, w3 = 1;
    int t = 3;
    int y, out;
    printf("Operasi AND dengan 3 Masukan\n");
    printf("Input X1 = ");
    scanf("%d", &x1);
    printf("Input X2 = ");
    scanf("%d", &x2);
    printf("Input X3 = ");
    scanf("%d", &x3);
    y = x1 * w1 + x2 * w2 + x3 * w3;
    if (y >= t)
        out = 1;
    else
        out = 0;
    printf("Output = %d\n", out);
    return 0;
}

```

Hasil:

a. Input x1 = 0, x2 = 0, x3 = 0

```
Operasi AND dengan 3 Masukan
Input X1 = 0
Input X2 = 0
Input X3 = 0
Output = 0
```

b. Input x1 = 0, x2 = 0, x3 = 1

```
Operasi AND dengan 3 Masukan
Input X1 = 0
Input X2 = 0
Input X3 = 1
Output = 0
```

c. Input x1 = 0, x2 = 1, x3 = 0

```
Operasi AND dengan 3 Masukan
Input X1 = 0
Input X2 = 1
Input X3 = 0
Output = 0
```

d. Input x1 = 0, x2 = 1, x3 = 1

x1	x2	x3	Output
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

```
Operasi AND dengan 3 Masukan
Input X1 = 0
Input X2 = 1
Input X3 = 1
Output = 0
```

e. Input x1 = 1, x2 = 0, x3 = 0

```
Operasi AND dengan 3 Masukan
Input X1 = 1
Input X2 = 0
Input X3 = 0
Output = 0
```

f. Input x1 = 1, x2 = 0, x3 = 1

```
Operasi AND dengan 3 Masukan
Input X1 = 1
Input X2 = 0
Input X3 = 1
Output = 0
```

g. Input x1 = 1, x2 = 1, x3 = 0

```
Operasi AND dengan 3 Masukan
Input X1 = 1
Input X2 = 1
Input X3 = 0
Output = 0
```

h. Input x1 = 1, x2 = 1, x3 = 1

```
Operasi AND dengan 3 Masukan
Input X1 = 1
Input X2 = 1
Input X3 = 1
Output = 1
```

2. Logika OR 3 Masukan

```c
#include <stdio.h>
int main() {
    int x1, x2, x3;
    int w1 = 2, w2 = 2, w3 = 2;
    int t = 2;
    int y, out;
    printf("Operasi OR dengan 3 Masukan\n");
    printf("Input X1 = ");
    scanf("%d", &x1);
    printf("Input X2 = ");
    scanf("%d", &x2);
    printf("Input X3 = ");
    scanf("%d", &x3);
    y = x1 * w1 + x2 * w2 + x3 * w3;
    if (y >= t)
        out = 1;
    else
        out = 0;
    printf("Output = %d\n", out);
    return 0;
}
```

Hasil:

a. Input x1 = 0, x2 = 0, x3 = 0

```
Operasi OR dengan 3 Masukan
Input X1 = 0
Input X2 = 0
Input X3 = 0
Output = 0
```

b. Input x1 = 0, x2 = 0, x3 = 1

```
Operasi OR dengan 3 Masukan
Input X1 = 0
Input X2 = 0
Input X3 = 1
Output = 1
```

c. Input x1 = 0, x2 = 1, x3 = 0

```
Operasi OR dengan 3 Masukan
Input X1 = 0
Input X2 = 1
Input X3 = 0
Output = 1
```

d. Input x1 = 0, x2 = 1, x3 = 1

```
Operasi OR dengan 3 Masukan
Input X1 = 0
Input X2 = 1
Input X3 = 1
Output = 1
```

e. Input x1 = 1, x2 = 0, x3 = 0

```
Operasi OR dengan 3 Masukan
Input X1 = 1
Input X2 = 0
Input X3 = 0
Output = 1
```

f. Input x1 = 1, x2 = 0, x3 = 1

```
Operasi OR dengan 3 Masukan
Input X1 = 1
Input X2 = 0
Input X3 = 1
Output = 1
```

g. Input x1 = 1, x2 = 1, x3 = 0

```
Operasi OR dengan 3 Masukan
Input X1 = 1
Input X2 = 1
Input X3 = 0
Output = 1
```

h. Input x1 = 1, x2 = 1, x3 = 1

```
Operasi OR dengan 3 Masukan
Input X1 = 1
Input X2 = 1
Input X3 = 1
Output = 1
```

x1	x2	x3	Output
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

3. Logika XOR 3 Masukan

```c
#include <stdio.h>
int xor_gate(int x1, int x2) {
    int w1 = 2, w2 = -1, w3 = 2, w4 = -1, w5 = 2, w6 = 2, t = 2;
    int y, o1, o2, o3, z1, z2;
    o1 = w1 * x1 + w4 * x2;
    z1 = (o1 >= t) ? 1 : 0;
    o2 = w2 * x1 + w3 * x2;
    z2 = (o2 >= t) ? 1 : 0;
    o3 = w5 * z1 + w6 * z2;
    y = (o3 >= t) ? 1 : 0;
    return y;
}
int main() {
    int x1, x2, x3, _temp;
    printf("Operasi XOR 3 Masukan\n");
    printf("Input x1 = ");
    scanf("%d", &x1);
    printf("Input x2 = ");
    scanf("%d", &x2);
    printf("Input x3 = ");
    scanf("%d", &x3);
    _temp = xor_gate(x1, x2);
    printf("Output = %d\n", xor_gate(_temp, x3));
    return 0;
}
```

Hasil:

a. Input x1 = 0, x2 = 0, x3 = 0

```
Operasi XOR 3 Masukan
Input x1 = 0
Input x2 = 0
Input x3 = 0
Output = 0
```

b. Input x1 = 0, x2 = 0, x3 = 1

```
Operasi XOR 3 Masukan
Input x1 = 0
Input x2 = 0
Input x3 = 1
Output = 1
```

c. Input x1 = 0, x2 = 1, x3 = 0

```
Operasi XOR 3 Masukan
Input x1 = 0
Input x2 = 1
Input x3 = 0
Output = 1
```

d. Input x1 = 0, x2 = 1, x3 = 1

x1	x2	x3	Output
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

```
Operasi XOR 3 Masukan
Input x1 = 0
Input x2 = 1
Input x3 = 1
Output = 0
```

e. Input x1 = 1, x2 = 0, x3 = 0

```
Operasi XOR 3 Masukan
Input x1 = 1
Input x2 = 0
Input x3 = 0
Output = 1
```

f. Input x1 = 1, x2 = 0, x3 = 1

```
Operasi XOR 3 Masukan
Input x1 = 1
Input x2 = 0
Input x3 = 1
Output = 0
```

g. Input x1 = 1, x2 = 1, x3 = 0

```
Operasi XOR 3 Masukan
Input x1 = 1
Input x2 = 1
Input x3 = 0
Output = 0
```

h. Input x1 = 1, x2 = 1, x3 = 1

```
Operasi XOR 3 Masukan
Input x1 = 1
Input x2 = 1
Input x3 = 1
Output = 1
```

F. Analisa

Pada praktikum ini, dilakukan implementasi gerbang logika (logic gates) AND, OR, AND NOT, dan XOR menggunakan bahasa pemrograman C++. Pada percobaan gerbang logika, setiap program menerima dua input biner, yaitu x1 dan x2, lalu menghitung nilai keluaran berdasarkan bobot (w) dan ambang batas (t).

1. Program AND dengan 2 input menggunakan bobot w1=1, w2=1, serta threshold t=2. Operasi dilakukan dengan menjumlahkan hasil perkalian bobot dengan input, lalu dibandingkan dengan threshold. Jika hasilnya lebih besar atau sama dengan t, maka output adalah 1, sebaliknya 0. Hasilnya sesuai dengan tabel kebenaran gerbang AND, di mana hanya 1 AND 1 yang menghasilkan 1.

```
Operasi AND
Input x1=0
Input x2=0
Output=0
```

```
Operasi AND
Input x1=0
Input x2=1
Output=0
```

```
Operasi AND
Input x1=1
Input x2=0
Output=0
```

```
Operasi AND
Input x1=1
Input x2=1
Output=1
```

2. Program OR dengan 2 input bekerja dengan cara serupa, tetapi menggunakan bobot w1=2 dan w2=2, dengan threshold t=2. Perbedaan utama adalah nilai bobot yang lebih besar, sehingga operasi OR menghasilkan output 1 jika setidaknya salah satu input bernilai 1. Hasil program

sesuai dengan tabel kebenaran OR, di mana 0 OR 0 adalah satu-satunya kombinasi yang menghasilkan 0.

```
Operasi OR
Input x1=0
Input x2=0
Hasil=0
```

```
Operasi OR
Input x1=0
Input x2=1
Hasil=1
```

```
Operasi OR
Input x1=1
Input x2=0
Hasil=1
```

```
Operasi OR
Input x1=1
Input x2=1
Hasil=1
```

3. Program AND NOT dengan 2 input merupakan variasi dari gerbang AND yang digabungkan dengan operasi NOT pada salah satu inputnya. Dalam implementasinya, digunakan bobot W1 = 2 dan W2 = -1, dengan threshold (t) sebesar 2. Perhitungan dilakukan dengan menjumlahkan hasil perkalian input dengan bobot masing-masing. Jika hasil perhitungan lebih besar atau sama dengan threshold, maka output bernilai 1, jika tidak, output bernilai 0. Dalam kasus ini, input X2 berperan sebagai faktor pembalik karena bobotnya negatif, sehingga hanya ketika X1 bernilai 1 dan X2 bernilai 0, output akan bernilai 1. Dalam kombinasi input lainnya, output bernilai 0.

```
Operasi AND NOT
Input x1=0
Input x2=0
Output=0
```

```
Operasi AND NOT
Input x1=0
Input x2=1
Output=0
```

```
Operasi AND NOT
Input x1=1
Input x2=0
Output=1
```

```
Operasi AND NOT
Input x1=1
Input x2=1
Output=0
```

4. Program XOR dengan 2 input lebih kompleks karena XOR bukan fungsi linear dan memerlukan dua lapisan keputusan. Program ini menggunakan dua unit tersembunyi dengan bobot berbeda untuk membentuk dua keputusan awal (out1 dan out2), yang kemudian diproses lagi untuk mendapatkan output akhir (out3). Struktur bobot dan threshold memastikan bahwa hanya kombinasi 1 XOR 0 dan 0 XOR 1 yang menghasilkan 1, sementara 0 XOR 0 dan 1 XOR 1 tetap 0. Dengan pendekatan ini, hasil yang diperoleh dari program XOR sesuai dengan tabel kebenaran XOR.

```
Operasi XOR
Input x1=0
Input x2=0
Output=0
```

```
Operasi XOR
Input x1=0
Input x2=1
Output=1
```

```
Operasi XOR
Input x1=1
Input x2=0
Output=1
```

```
Operasi XOR
Input x1=1
Input x2=1
Output=0
```

5. Program AND dengan 3 input ini, logika pada program ini diperluas dengan menambahkan satu input tambahan dengan sebelumnya, yaitu X3. Sama seperti pada AND 2 input, setiap nilai input dikalikan dengan bobot 1 dan hasilnya dijumlahkan. Jika hasilnya lebih besar atau sama dengan 3 (karena ada tiga input), maka output bernilai 1. Dengan demikian, output hanya akan bernilai 1 jika ketiga input bernilai 1, sedangkan kombinasi lain akan menghasilkan output 0.
6. Program OR dengan 3 input ini, prinsipnya serupa dengan OR 2 input tetapi dengan penambahan satu variabel input. Bobot W1, W2, dan W3 tetap memiliki nilai 2. Jika hasil perhitungan mencapai minimal 2, maka output bernilai 1. Dengan demikian, selama ada minimal satu input yang bernilai 1, maka output akan menjadi 1. Hanya jika ketiga input bernilai 0, maka outputnya adalah 0.
7. Program XOR dengan 3 input memiliki perhitungan lebih kompleks dibandingkan versi 2 input. Seperti sebelumnya, setiap input dikalikan dengan bobot tertentu, lalu hasilnya dikalikan dengan faktor tambahan. Jika hasil akhir lebih besar atau sama dengan threshold, maka output bernilai 1. Pada XOR 3 input, karakteristiknya tetap bahwa output hanya bernilai 1 jika jumlah input bernilai 1 adalah ganjil (misalnya 1-0-0, 0-1-0, atau 1-1-1), sedangkan jika jumlahnya genap (misalnya 0-0-0 atau 1-1-0), outputnya akan bernilai 0.

G. Kesimpulan

Program aplikasi McCulloch-Pitts berhasil divisualisasikan dalam bentuk implementasi gerbang logika menggunakan operasi matematika berbasis bobot dan threshold yang menghasilkan output sesuai tabel kebenaran. Percobaan ini mencakup gerbang AND, OR, AND NOT, dan XOR dengan 2 dan 3 input, di mana setiap input dikalikan dengan bobot tertentu dan dibandingkan dengan ambang batas (threshold) untuk menentukan output. Program AND hanya menghasilkan output 1 jika semua input bernilai 1, sedangkan OR menghasilkan 1 jika minimal satu input bernilai 1. AND NOT memanfaatkan bobot negatif untuk membalik salah satu input, sehingga hanya kombinasi tertentu yang menghasilkan output 1. Sementara itu, XOR memiliki perhitungan lebih kompleks karena memerlukan dua lapisan keputusan untuk memastikan bahwa output hanya bernilai 1 jika jumlah input bernilai 1 adalah ganjil. Dengan pendekatan ini, seluruh program telah berhasil meniru perilaku dasar gerbang logika sesuai dengan teori McCulloch-Pitts.